

Document number: P0824R1

Date: 2018-02-05

Project: ISO JTC1/SC22/WG21, Programming Language C++

Audience: Library Evolution Working Group

Reply to: Arthur O'Dwyer <arthur.j.odwyer@gmail.com>, Charley Bay <charleyb123@gmail.com>, Odin Holmes <holmes@auto-intern.de>, Michael Wong <fraggamuffin@gmail.com>, Niall Douglas <s_sourceforge@nedprod.com>

Summary of SG14 discussion on <system_error>

[1. Introduction](#)

[2. Description of C++11's <system_error> facilities](#)

[3. Proposed best practices for using C++11's <system_error> facilities](#)

[3.1. Proposed by Arthur O'Dwyer](#)

[3.2. Proposed by Charley Bay](#)

[4. Issues with the <system_error> facilities](#)

[4.1. Use of std::string](#)

[4.2. Proliferation of "two-API" libraries](#)

[4.3. No wording sets aside the 0 enumerator](#)

[4.4. Reliance on singletons](#)

[4.5. No error_category subclass can be a literal type](#)

[4.6. No guidance on attaching extra information to error_code](#)

[4.7. Reliance on a surprising overload of operator==](#)

[4.8. error_category should properly be named error_domain](#)

[4.9. Standard error_code-yielding functions can throw exceptions anyway](#)

[4.10. Underspecified error_code comparison semantics](#)

1. Introduction

This paper summarizes a discussion that took place during the SG14 telecon of 2017-10-11. The discussion concerned the facilities of <system_error> (introduced in C++11), which include `errc`, `error_code`, `error_condition`, `error_category`, and `system_error`.

The discussion naturally also concerns the current idioms for exceptionless "disappointment handling" ([P157R0](#), Lawrence Crowl), as seen in <filesystem> (C++17) and `std::from_chars/to_chars` (also C++17); and it concerns *future* idioms for exceptionless disappointment handling, as proposed in `status_value<E,T>` ([P0262R0](#), Lawrence Crowl), in `expected<T,E>` ([P0323R2](#), Vicente J. Botet Escribá), and in `result<T,EC>` ([Outcome](#), Niall Douglas).

In all of those future idioms involving `expected<T,E>`-style result types, SG14 expects that the `E` type parameter will default to `std::error_code` — which is right and good. This means that the coming years will see much increased use of `std::error_code` in both new and existing codebases. This means that SG14 is very interested in correcting deficiencies in `std::error_code` sooner, rather than later.

On SG14's 2017-10-11 teleconference call, various people contributed to a miscellaneous "laundry list" of perceived deficiencies with <system_error>. Several "best practices" were also proposed for how we expect <system_error> facilities to be used in the best codebases. (Unfortunately, the Standard Library does not follow *anyone's* proposed "best practice"!)

This paper is a summary of that discussion.

2. Description of C++11's <system_error> facilities

<system_error> provides the following patterns for dealing with error codes:

A `std::error_code` object wraps an integer "error enumerator" and a pointer to an "error category" (that is, a handle to the specific error domain associated with this integer enumerator). The addition of the error domain handle is what allows us to distinguish, say, `error=5` "no leader for Kafka partition" (in the `rdkafka` domain) from `error=5` "I/O error" (in the POSIX domain). Two `error_code` instances compare equal if and only if they represent the same error enumerator *in the same domain*.

Notice that `std::error_condition` is not the same type as `std::error_code`!

```
namespace std {
class error_code {
    int value_;
    std::error_category *cat_;
public:
    error_code() noexcept : error_code(0, std::system_category()) {}
    error_code(int e, const std::error_category& c) noexcept : value_(e), cat_(&c) {}
    template<class E> error_code(E e) noexcept requires(std::is_error_code_enum_v<E>) { *this = make_error_code(e); } // intentional ADL
    template<class E> error_code& operator=(E e) noexcept requires(std::is_error_code_enum_v<E>) { *this = make_error_code(e); } // inter
    void assign(int e, const std::error_category& c) noexcept { value_ = e; cat_ = &c; }
    void clear() noexcept { *this = std::error_code(); }

    explicit operator bool() const noexcept { return value_ != 0; }
    int value() const noexcept { return value_; }
    const std::error_category& category() const noexcept { return cat_; }

    std::string message() const { return cat_->message(value_); }
    std::error_condition default_error_condition() const noexcept { return cat_->default_error_condition(value_); }
};
bool operator==(const std::error_code& a, const std::error_code& b) noexcept { return a.value() == b.value() && &a.category() == &b.categc
} // namespace std
```

Each "error domain" is represented in code by an object of type `std::error_category`. The standard effectively implies that these must be singletons, because `error_code` equality-comparison is implemented in terms of pointer comparison (see above). So two `error_category` objects located at different memory addresses represent different error domains, *by definition*, as far as the currently standardized scheme is concerned.

```
namespace std {
class error_category {
public:
    constexpr error_category() noexcept = default;
```

```

    error_category(const error_category&) = delete;
    virtual ~error_category() {}

    virtual const char *name() const noexcept = 0;

    virtual std::error_condition default_error_condition(int e) const noexcept { return std::error_condition(e, *this); }
    virtual std::string message(int e) const = 0;

    virtual bool equivalent(int e, const std::error_condition& condition) const noexcept { return this->default_error_condition(e) == condition; }
    virtual bool equivalent(const std::error_code& code, int e) const noexcept { return code == std::error_code(e, *this); }
};
bool operator==(const std::error_category& a, const std::error_category& b) noexcept { return &a == &b; }
} // namespace std

```

It is intended that the programmer should *inherit publicly* from `std::error_category` and override its pure virtual methods (and optionally override its non-pure virtual methods) to create new library-specific error domains. An error domain encompasses a set of error enumerators with their associated human meanings (for example, `error=5` meaning "no leader for Kafka partition"). So for example we can expect that `rdkafka_category().message(5)` would return `std::string("no leader for partition")`.

The standard also provides a class `std::error_condition` which is almost identical in implementation to `std::error_code`.

```

namespace std {
class error_condition {
    int value_;
    std::error_category *cat_;
public:
    error_condition() noexcept : error_code(0, std::generic_category()) {}
    error_condition(int e, const std::error_category& c) noexcept : value_(e), cat_(&c) {}
    template<class E> error_condition(E e) noexcept requires(std::is_error_condition_enum_v<E>) { *this = make_error_condition(e); } // i
    template<class E> error_condition& operator=(E e) noexcept requires(std::is_error_condition_enum_v<E>) { *this = make_error_condition(e); }
    void assign(int e, const std::error_category& c) noexcept { value_ = e; cat_ = &c; }
    void clear() noexcept { *this = std::error_condition(); }

    explicit operator bool() const noexcept { return value_ != 0; }
    int value() const noexcept { return value_; }
    const std::error_category& category() const noexcept { return cat_; }

    std::string message() const { return cat_->message(value_); }
};
bool operator==(const std::error_condition& a, const std::error_condition& b) noexcept { return a.value() == b.value() && &a.category() == &b.category(); }
} // namespace std

```

The best practice for using `std::error_condition` is the subject of some debate in SG14; see the rest of this paper. However, clearly the vague intent of `std::error_condition` is to represent in some way a high-level "condition" which a low-level "error code" (thrown up from the bowels of the system) might or might not "match" in some high-level semantic sense. Notice that the error domain of a default-constructed `error_code` is `system_category()`, whereas the error domain of a default-constructed `error_condition` is `generic_category()`.

The standard provides a highly customizable codepath for comparing an `error_code` against an `error_condition` with `operator==`. (Both `error_code` and `error_condition` are value types — in fact they are trivial types — which means that their own `operator==(A,A)` are just bitwise comparisons. We are now speaking of `operator==(A,B)`.)

```

namespace std {
bool operator==(const std::error_code& a, const std::error_condition& b) noexcept {
    return a.category().equivalent(a.value(), b) || b.category().equivalent(a, b.value());
}
bool operator==(const std::error_condition& a, const std::error_code& b) noexcept {
    return a.category().equivalent(a.value(), b) || b.category().equivalent(a, b.value());
}
} // namespace std

```

Recall that the base-class implementation of `error_category::equivalent()` is just to compare for strict equality; but the programmer's own `error_category`-derived classes can override `equivalent()` to have different behavior.

Lastly, the `<system_error>` header provides an exception class that wraps an `error_code` (but not an `error_condition`):

```

namespace std {
class system_error : public runtime_error {
    std::error_code code_;
    std::string what_;
public:
    system_error(std::error_code ec) : code_(ec), what_(ec.message()) {}
    system_error(std::error_code ec, const std::string& w) : code_(ec), what_(ec.message() + ": " + w) {}
    system_error(std::error_code ec, const char *w) : system_error(ec, std::string(w)) {}
    system_error(int e, const std::error_category& cat) : system_error(std::error_code(e, cat)) {}
    system_error(int e, const std::error_category& cat, const std::string& w) : system_error(std::error_code(e, cat), w) {}
    system_error(int e, const std::error_category& cat, const char *w) : system_error(std::error_code(e, cat), w) {}
    const std::error_code& code() const noexcept { return code_; }
    const char *what() const noexcept override { return what_.c_str(); }
};
} // namespace std

```

Note that `std::filesystem::filesystem_error` inherits from `std::system_error`.

3. Proposed best practices for using C++11's `<system_error>` facilities

3.1. Proposed by Arthur O'Dwyer

In the example that follows, we have an application `appA`, which calls into library `libB`, which calls into library `libC`, which calls into library `libD`.

Arthur O'Dwyer proposes the following general rules:

1. No enumeration type `E` should ever satisfy *both* `is_error_code_enum_v<E>` and `is_error_condition_enum_v<E>` simultaneously. (To do so would be to represent a low-level error code value and a high-level abstract condition simultaneously, which is impossible.)
2. For any enumeration type `E`, the ADL function `make_error_code(E)` should exist if-and-only-if `is_error_code_enum_v<E>`; and the ADL function `make_error_condition(E)` should exist if-and-only-if `is_error_condition_enum_v<E>`.
3. In any enumeration type `E` satisfying either `is_error_code_enum_v<E>` or `is_error_condition_enum_v<E>`, the enumerator value `0` must be set aside as a "success" value, and never allotted to any mode of failure. In fact, the enumeration `E` should have an enumerator `success = 0` or `none = 0` to ensure that this invariant is never broken accidentally by a maintainer.
4. Your library should have exactly as many `error_category` subclasses as it has enumeration types satisfying either `is_error_code_enum_v<E>` or `is_error_condition_enum_v<E>`, in a one-to-one correspondence. No `error_category` subclass should be "shared" between two enumeration types; and no `error_category` subclass should exist that is not associated with a specific enumeration type.
5. Each `error_category` subclass should be a singleton; that is, it should have a single instance across the entire program.
6. When your library detects a failure, it should construct an `std::error_code` representing the failure. This can be done using ADL `make_error_code(LibD::ErrCode::failure_mode)` or simply using `LibD::ErrCode::failure_mode` (which works because of `std::error_code`'s implicit constructor from error-code-enum types).
7. This `std::error_code` is passed up the stack using out-parameters (as `<filesystem>`) or using `Expected<T, std::error_code>`.
8. When `libB` receives a `std::error_code` code that must be checked for *failure versus success*, it should use `if (code)` or `if (!code)`.
9. When `libB` receives a `std::error_code` code that must be checked for *a particular source-specific error* (such as "rdkafka partition lacks a leader"), it should use `if (code == LibD::ErrCode::failure_mode)`. This performs exact equality, and is useful if you know the exact source of the error you're looking for (such as "rdkafka").
10. When `libB` receives a `std::error_code` code that must be checked for *a high-level condition* (such as "file not found"), which might correspond to any of several source-specific errors across different domains, it may use `if (code == LibC::ErrCondition::failure_mode)`, where `LibC::ErrCondition` is an error-condition-enum type provided by the topmost library (the one whose API we're calling — not any lower-level library). This will perform semantic classification.
11. Your library might perhaps *provide* semantic classification by providing an error-condition-enum type `LibB::ErrCondition` (and its associated `error_category` subclass `LibB::ErrConditionCategory`), which encodes knowledge about the kinds of error values reported by `LibC`. Ideally, `LibB::ErrConditionCategory::equivalent()` should defer to `LibC::ErrConditionCategory::equivalent()` in any case where `LibB` is unsure of the meaning of a particular error code (for example, if it comes from an unrecognized error domain).
12. Most likely, `std::error_condition` and error-condition-enum types should simply *not be used*. `libB` should not expect its own callers to write `if (code == LibB::ErrCondition::oom_failure)`; instead `libB` should expect its callers to write `if (LibB::is_oom_failure(code))`, where `bool LibB::is_oom_failure(std::error_code)` is a free function provided by `libB`. This successfully accomplishes semantic classification, and does it without any operator overloading, and therefore does it without the need for the `std::error_condition` type.

3.2. Proposed by Charley Bay

In the example that follows, we have an application `appA`, which calls into library `libB`, which calls into library `libc`, which calls into library `libD`.

Charley Bay proposes (something like) the following general rules (paraphrased here by Arthur O'Dwyer):

1. No enumeration type `E` should ever satisfy *both* `is_error_code_enum_v<E>` and `is_error_condition_enum_v<E>` simultaneously. (To do so would be to represent a low-level error code value and a high-level abstract condition simultaneously, which is impossible.)
2. For any enumeration type `E`, the ADL function `make_error_code(E)` should exist if-and-only-if `is_error_code_enum_v<E>`; and the ADL function `make_error_condition(E)` should exist if-and-only-if `is_error_condition_enum_v<E>`.
3. In any enumeration type `E` satisfying either `is_error_code_enum_v<E>` or `is_error_condition_enum_v<E>`, the enumerator value `0` must be set aside as a "success" value, and never allotted to any mode of failure. In fact, the enumeration `E` should have an enumerator `success = 0` or `none = 0` to ensure that this invariant is never broken accidentally by a maintainer.
4. `error_category` subclasses are not necessarily singletons. It is conceivable that multiple instances of the same `error_category` subclass type could exist within the same program.
5. When your library detects a failure, it should construct an `std::error_code` representing the failure. This should be done using `LibD::ErrCode::failure_mode` (which works because of `std::error_code`'s implicit constructor from error-code-enum types).
6. This `std::error_code` is passed up the stack using out-parameters (as `<filesystem>`) or using `Expected<T, std::error_code>`.
7. When `libB` receives a `std::error_code` code, it must *never* be checked for *a particular source-specific error* (such as "rdkafka partition lacks a leader"). Every test should be done on the basis of semantic classification — whether at the coarse granularity of "failure versus success" or at the fine granularity of "partition lacks a leader." It is always conceivable that `libc` might change out its implementation so that it no longer uses `libD`; therefore, all testing of error codes returned by a `libc` API must be expressed in terms of the specific set of abstract failure modes exposed by that same `libc` API.
8. When `libB` receives a `std::error_code` code, it should *not* be checked for *failure versus success* via `if (code)` or `if (!code)`. Instead, "failure" should be a semantic classification as described in the preceding point.
9. As explained in the preceding point, exact-equality comparisons should never be used. But with the standard syntax, there is a significant risk that the programmer will accidentally write `if (code == LibB::ErrCode::oom_failure)` (exact-equality comparison) instead of the intended `if (code == make_error_condition(LibB::ErrCode::oom_failure))` (semantic classification). Therefore, under the current standard library design, `std::error_condition` and error-condition-enum types should *not be used*. `libB` should expect its callers to write `if (LibB::is_oom_failure(code))`, where `bool LibB::is_oom_failure(std::error_code)` is a free function provided by `libB`. This successfully accomplishes semantic classification, and does it without any operator overloading, and therefore does it without the need for the `std::error_condition` type.

4. Issues with the `<system_error>` facilities

On the 2017-10-11 teleconference, the following issues were discussed.

4.1. Use of `std::string`

`std::error_category`'s method `virtual std::string message(int) const` converts an error-code-enum into a human-readable message. This functionality is apparently useful only for presentation to humans; i.e., we do not expect that anyone should be treating the result of `message()` as a unique key, nor scanning into its elements with `strstr`. However, as a pure virtual method, this method *must* be implemented by each `error_category` subclass.

Its return type is `std::string`, i.e., it introduces a hard-coded dependency on `std::allocator<char>`. This seems to imply that if you are on a computer system without `new` and `delete`, without `std::allocator`, without `std::string`, then you cannot implement your own `error_category` subclasses, which effectively means that you cannot use `std::error_code` to deal with disappointment. SG14 sees any hard-coded dependency on `std::allocator` as unfortunate. For a supposedly "fundamental" library like `<system_error>` it is *extremely* unfortunate. (LWG issue 2955 is related: `std::from_chars` used to depend on `std::error_code` and thence on `std::string`. It was resolved by the adoption of P0682R1, i.e., `std::from_chars` simply stopped trying to use `std::error_code` at all.)

During the SG14 telecon, Charley Bay commented that returning ownership of a dynamically allocated string allows the `error_category` subclass to return a message that differs based on the *current locale*. However, nobody on the call claimed that this functionality was important to them. Furthermore, if locale-awareness were desirable, then branching on the *current (global) locale* would be the wrong way to go about it, because that mechanism would not be usable by multi-threaded programs. The right way to introduce locale-awareness into `error_category` would be to provide a virtual method `std::string message(int, const std::locale&)`.

SG14 seems to agree that eliminating the dependency on `std::allocator` would be nice.

SG14 seems to agree that dynamically allocated message strings are not an important feature.

Two ways of removing `std::string` were proposed: return `const char*`, or return `std::string_view`. Arthur O'Dwyer commented that he strongly prefers `const char*` for simplicity (no new library dependencies) and for consistency with `std::error_category::name()` and `std::exception::what()`. Niall Douglas commented that he prefers `std::string_view` over raw null-terminated `const char*` whenever possible.

Both ways of removing `std::string` alter the return type of a pure virtual method and thus inevitably break every subclass of `std::error_category` ever. SG14 has no way out of this dilemma other than to suggest "wait for std2 and do it correctly there."

4.2. Proliferation of "two-API" libraries

`<filesystem>` is the poster child for this issue. Every function and method in `<filesystem>` comes in two flavors: throwing and non-throwing. The throwing version gets the "natural" signature (as is right and expected in C++); and the non-throwing version gets a signature with an extra out-parameter of type `std::error_code&`. The expectation is apparently that `<system_error>` users will be willing to write "C-style" code:

```
namespace fs = std::filesystem;
void truncate_if_large(const fs::path& p) noexcept
{
    std::error_code ec; // declare an uninitialized variable
    uintmax_t oldsize = fs::file_size(p, ec);
    if (ec) { report_error(ec); return; }
    if (oldsize > 1000) {
        fs::resize_file(p, 1000, ec);
        if (ec) { report_error(ec); return; }
    }
}
```

It would be nicer if the non-throwing API had exactly the same signatures as the throwing API, except that it should return `expected<T>` or `result<T>` instead of `T`. On the telecon, Arthur O'Dwyer commented that this can't easily be done because you cannot have two functions with the same name and the same signature, differing only in return type.

It is possible to segregate the *free functions* into a separate namespace, say `namespace std::filesystem::nothrow`, so that the above code could be written as

```
namespace fs = std::filesystem::nothrow; // hypothetical
void truncate_if_large(const fs::path& p) noexcept
{
    auto oldsize = fs::file_size(p);
    if (!oldsize.has_value()) { report_error(oldsize.error()); return; }
    if (oldsize.value() > 1000) {
        auto failure = fs::resize_file(p, 1000);
        if (failure) { report_error(failure.error()); return; }
    }
}
```

However, this doesn't help with member functions, such as `directory_entry::is_symlink()`.

Having two APIs (throwing and non-throwing) side by side in the same namespace has another disadvantage. There is a significant risk that the programmer might accidentally leave off the out-parameter that signifies "non-throwing-ness", resulting in a call to the throwing version when a call to the non-throwing version was intended.

```
namespace fs = std::filesystem;
void truncate_if_large(const fs::path& p) noexcept
{
    std::error_code ec; // declare an uninitialized variable
    uintmax_t oldsize = fs::file_size(p, ec);
    if (ec) { report_error(ec); return; }
    if (oldsize > 1000) {
        fs::resize_file(p, 1000); // Oops! Bug goes undetected by all major vendors.
        if (ec) { report_error(ec); return; }
    }
}
```

The LLVM project has already observed this failure mode happening in the wild ([bug identified](#), [bugfix commit](#)). It is desirable to clearly segregate throwing from non-throwing functions. But we don't know how to make segregation work for member functions. Therefore perhaps the *best* outcome would be to stick with a single (non-throwing) API for each library.

If we had a single (non-throwing) API that returned something like `Expected<T>`, and if `Expected<T>` had a member function `T or_throw()` that returned the `ex.value()` if possible or else threw a `system_error` initialized from `ex.error()`, then we could write exception-throwing code fluently as follows:

```
namespace fs = std::filesystem::nothrow;
void truncate_if_large(const fs::path& p) noexcept
{
    uintmax_t oldsize = fs::file_size(p).or_throw();
    if (oldsize > 1000) {
```

```

        fs::resize_file(p, 1000).or_throw();
    }
}

```

Here we assume that the template class `Expected<void>` is marked with the standard `[[nodiscard]]` attribute, so that if the programmer accidentally leaves off the final `or_throw()` the compiler will emit a warning.

SG14 seems not to have a great answer for how to avoid "two-API" libraries such as `<filesystem>` going forward; but we believe that "two-API" libraries should be avoided. The Networking TS seems to be shaping up to be another "two-API" library. We believe this is unfortunate.

4.3. No wording sets aside the 0 enumerator

The current Standard strongly implies the best-practice mentioned above: that every error-code-enumerator and every error-condition-enumerator should set aside `success = 0` as a special case.

```

enum class TroublesomeCode { out_of_memory, out_of_files };
struct TroublesomeCategory : public std::error_category {
    const char *name() const noexcept override { return ""; }
    std::string message(int e) const override {
        switch (e) {
            case TroublesomeCode::out_of_memory: return "out of memory";
            case TroublesomeCode::out_of_files: return "out of files";
            default: __builtin_unreachable();
        }
    }
};
const std::error_category& troublesome_category() {
    static const TroublesomeCategory instance;
    return instance;
}

template<> struct std::is_error_code_enum<TroublesomeCode> : std::true_type {};
std::error_code make_error_code(TroublesomeCode e) {
    return std::error_code((int)e, troublesome_category());
}

int main() {
    std::error_code ec = TroublesomeCode::out_of_memory;
    if (ec) {
        puts("This line will not be printed.");
    }
}

```

If the current specification of `std::error_code` is to remain untouched, then SG14 would like to see some explicit acknowledgment in the Standard that error-code enumerators with value 0 are "special," i.e., they will not be treated as "errors" by any of the machinery in the Standard. Error codes with value 0 are effectively reserved for the "success" case, and programmers should not attempt to use them for any other purpose.

Vice versa, programmers should be aware that using a non-zero integer value to represent "success" will not work as expected. Consider an HTTP library that naively attempts to use `ok = 200` as its "success" code, and then provides an ADL `make_error_code` like this:

```

enum class HTTPStatusCode { ok = 200, created = 201, /* ... */ };

template<> struct std::is_error_code_enum<HTTPStatusCode> : std::true_type {};
std::error_code make_error_code(HTTPStatusCode e) {
    return std::error_code((e == ok) ? 0 : (int)e, http_status_category());
}

std::string HTTPStatusCategory::message(int e) const {
    switch (e) {
        case 0: return "200 OK";
        case 201: return "201 Created";
        // ...
    }
}

```

The programmer may head far down this "garden path" under the assumption that his goal of a non-zero "ok" code is attainable; but we on the Committee know that it is *not* attainable. We should save the programmer some time and some headaches, by explicitly reserving error-code 0 in the standard.

However, there is an alternative and perhaps better solution, which is to replace certain constructors and member functions of `std::error_code` and `std::error_condition` as follows: Rather than

```

error_code() noexcept : error_code(0, std::system_category()) {}
error_condition() noexcept : error_code(0, std::generic_category()) {}
explicit operator bool() const noexcept { return value_ != 0; }

```

we could propose

```

error_code() noexcept : error_code(0, std::null_category()) {}
error_condition() noexcept : error_code(0, std::null_category()) {}
explicit operator bool() const noexcept { return cat_ != &std::null_category(); }

```

This would eliminate the current requirement that error-code 0 always be reserved. Providing a default "null category" could also help to clarify the expected semantics of a default-constructed `error_condition`, which at present is implementation-defined and varies by vendor.

4.4. Reliance on singletons

`std::error_category` implicitly relies on singletons. Even if the programmer can somehow get away with using non-singletons for his own categories, the standard library's own categories (e.g. `std::generic_category`) are singletons: their `operator==` is explicitly defined to compare instances for address-equality.

`std::error_category` is not the only standard C++ feature to rely on singletons: we have prior art in the form of the `std::type_info` singletons which are used by `dynamic_cast` and also by `catch`.

Prior to the SG14 telecon, Niall Douglas raised the point that singletons do not play well with DLLs (a.k.a. shared objects, a.k.a. dylibs). On some platforms, there are common programming idioms which can cause a C++ library's "singletons" to become duplicated. These include at least:

- Library A is statically linked with Boost v1.5; library B is statically linked with Boost v1.6; the application is statically linked with libraries A and B. All of Boost's singletons are duplicated. (In some cases this is actually the desired behavior; in other cases we'd actually want some of them merged together if we had the choice.)
- Library A is statically linked with Boost; library B is also statically linked with Boost; the application dynamically loads library A (with `RTLD_LOCAL` or the equivalent) and then dynamically loads library B. Library B cannot "see" library A's exported symbols, so it brings in duplicate copies of all the Boost singletons.

Arthur O'Dwyer's (admittedly uninformed) opinion is that these sound like antipatterns that could reasonably be avoided. Niall Douglas's opinion is that these are patterns in use in big consumers, e.g. Python loads its C/C++ modules with `RTLD_LOCAL`, and indeed has no choice but to do so. Charley Bay has also seen these failure modes in practice.

The practical difficulty of using singletons in DLLs seems to be a continuing pain point for certain programmers. The use of singletons by RTTI seems to be a continuing reason that some programmers avoid `dynamic_cast` and exception handling. The use of singletons by `std::error_category` will cause the same kinds of problems in practice as the use of singletons by RTTI in exception-handling. If we cannot figure out how to eliminate these practical problems, then `std::error_code` cannot possibly be a suitable replacement for exception-handling because it will continue to have the same problems (namely, a reliance on singletons compared for address-equality).

In practice, some platforms (notably MSVC, and `libc++` if built with a compile-time flag) work around the above problems for `type_info` singletons by implementing `type_info::operator==` as a string-equality (i.e. `strcmp(this->name(), rhs.name())`) instead of an address-equality (i.e. `this == &rhs`). This workaround is blessed by the Standard; `type_info::operator==` is specified to return "true if the two values describe the same type" with no constraints on *how* this "sameness" is determined. In contrast, `error_category::operator==` is specified to return exactly `"this == &rhs"` with no wiggle room at all.

SG14 suggests that perhaps `error_category::operator==` is overspecified, and that it could be relaxed to allow for string-equality comparison.

SG14 identifies "singletons in DLLs" as a problem area. It is currently unclear how to handle "singletons in DLLs" in C++. Whoever knows the best practice in this area should speak up.

4.5. No `error_category` subclass can be a literal type

During the SG14 telecon, Odin Holmes and Charley Bay raised the issue that `std::error_code` is not usable in `constexpr` contexts; for example, `bool operator==(const error_code& lhs, const error_code& rhs) noexcept` is not `constexpr`. Even constructing a `std::error_code` instance cannot be done `constexprly`, because `error_code(int val, const error_category& cat) noexcept` is not `constexpr`. Even if it *were* `constexpr`, we *still* wouldn't be able to construct an `error_code` `constexprly`, because we couldn't get the appropriate `const error_category&`, because for example `const error_category& generic_category() noexcept` is not `constexpr`!

It is impossible to manipulate any `error_category` instance `constexprly`, because `error_category` is not a literal type: it has a virtual destructor, whereas literal types require trivial destructors. This is especially unfortunate because `error_category` does not need polymorphic destruction.

During the SG14 telecon, Odin Holmes explained that in his programs he often uses `constexpr` on functions that he does *not* expect to be evaluated at compile-time. The reason he uses `constexpr` is to demonstrate and enforce that the functions are *pure*; and their compile-time-evaluability is just an occasional bonus. This usage of `constexpr` would have been infeasible in C++11, where `constexpr` functions were constrained to single statements; but it is feasible in C++14 and later. In C++11, the programmer must learn two different programming styles: a convenient and fluid style for "run-time" functions, and a highly convoluted and obfuscated style for "compile-time" `constexpr` functions. In C++14, the programmer can generally use a single, convenient, fluid style for both "run-time" functions and "compile-time" `constexpr` functions; the difference between a non-`constexpr` function and a `constexpr` function in C++14 is usually just the addition or subtraction of the keyword `constexpr`.

However, if a function *A*'s body unconditionally uses some non-`constexpr` function or constructs some non-literal type *B*, then adding the keyword `constexpr` to *A* will trigger a compiler error. Examples of non-`constexpr` functions and types include `std::vector` and `std::regex`... but also `std::error_code`!

```
constexpr int f(int i) {
    std::error_code ec;
    return i;
}

error: variable of non-literal type 'std::error_code' cannot be defined in a constexpr function
    std::error_code ec;
                      ^
```

Even if we forget about the potentially tricky code to raise and handle `error_codes`, and just try to propagate an error code up from the lower level to the higher level, we find that we cannot do it in a `constexpr` way. (This error message is from GCC with `libstdc++. libcpp` unilaterally adds `constexpr` to `error_code::operator bool()`, which is a conforming extension.)

```
constexpr int constexpr_4throot(int i, std::error_code& ec) {
    int j = constexpr_sqrt(i, ec);
    if (ec) return 0;
    return constexpr_sqrt(j, ec);
}

error: call to non-constexpr function 'std::error_code::operator bool() const'
    if (ec) return 0;
    ^
```

SG14 would like to develop some good idioms for error handling with `std::error_code`; but at present, these idioms (such as the `if (ec)` in the above code) cannot be used in `constexpr` functions. This sends C++ programmers back to the dark ages of C++11, where we need to learn two different styles of programming: a convenient, fluid style (using `error_code`) for "run-time" functions and a constrained, convoluted style (eschewing `error_code`) for functions we want to mark `constexpr`.

SG14 suggests that `std::error_category`'s destructor should originally have been non-virtual. It is too late to change `std::error_category` at this point, though, because removing the virtual specifier would noisily break idiomatic C++11 code such as the following:

```
struct my_category : public std::error_category {
    // ...
    ~my_category() override; // OK iff ~error_category is virtual
```

```
}
```

So it seems that `std::error_category` cannot be made constexpr-friendly. We have not investigated whether `std::error_code` itself can be made constexpr-friendly, but we would like to see some work in this area.

4.6. No guidance on attaching extra information to error_code

Niall Douglas has attempted to subclass `std::error_code` in order to create a kind of `extended_error_code` that contains not only a category pointer and an integer but also some kind of "payload", such as a string or variant holding the arguments of the operation that failed. This is very similar in intent to the C++17 standard library's `std::filesystem::filesystem_error`, which holds a `std::error_code` and two instances of `std::filesystem::path` holding the arguments of the operation that failed. However, `std::filesystem::filesystem_error` is a subclass of `std::runtime_error`, whereas Niall and Charley are trying to make something non-polymorphic to be used in the absence of exception-handling.

Niall's further experiments with "error_code plus payload" have resulted in a design he calls "status_code", described in [a thread on the SG14 reflector](#).

Charley Bay has attempted a similar goal via different means. Charley created a subclass of `std::error_category` which maintained a lookup table of the "payload" for each `error_code` currently in flight. (This lookup table needs some mechanism for "garbage-collection," since `error_code` objects are trivially destructible and have no built-in hook by which they could be reference-counted.)

The standard library does not currently provide an `extended_error_code` type. For people who need (or think they need) something along these lines, it's hard to tell whether they should be inheriting from `error_code` (will that lead to slicing pitfalls? overload resolution gaffes?), or aggregating à la `filesystem_error`, or simply passing the extra data around manually via a second out-parameter or something like `pair<error_code, string>`. Anyway, C++ does not provide a standard wheel, so different programmers may end up inventing slightly different wheels here.

4.7. Reliance on a surprising overload of operator==

The library design relies on the ability to perform a "semantic match" operation between a `std::error_code` (a low-level error indicator) and a `std::error_condition` (a high-level condition). The "semantic match" operation is expressed in C++ source code as `(ec == econd)`. During the SG14 telecon, Charley Bay remarked that this use of `operator==` is surprising because it represents an operation that is fundamentally *unlike* equality.

For example, "equality" is transitive, but "semantic match" is not necessarily transitive, in that we can have `ec1 == econd1 && econd1 == ec2 && ec1 != ec2` or `econd1 == ec1 && ec1 == econd2 && econd1 != econd2`.

Also, "semantic match" is not necessarily (notionally) symmetric or reflexive, in that we can construct examples where `make_error_code(e1) == make_error_condition(e2) && make_error_code(e2) != make_error_condition(e1)` or even `make_error_code(e1) != make_error_condition(e1)`. However, these examples require that the enumeration type of `e1` define both `make_error_code` and `make_error_condition`, which contradicts Arthur's suggested best practices.

It might have been better for the library's API to use a meaningful identifier such as `ec.matches(econd)`, rather than hijacking the `==` operator for this distinct semantic-match operation.

4.8. error_category should properly have been named error_domain

SG14 generally concluded that the best way to explain the intention of `error_category` is to say that each singleton derived from `error_category` represents a particular *domain* of errors. Each category object understands and manages error codes in a particular *domain* — for example, "POSIX errors" or "filesystem errors" or "rdkafka errors." It was generally concluded that the English word "domain" expresses this notion more clearly and appropriately than the English word "category."

It is obviously too late to change the name of `std::error_category` to `std::error_domain`; that ship has sailed. In the SG14 telecon it was remarked that maybe it's fortunate that the name `std::error_domain` is still available. If someone designs a "better error_category" (one without singletons, or without `std::string`, or with constexpr support), then the new thing can be called `std::error_domain` and we can stop teaching the "awkwardly named" `std::error_category`.

4.9. Standard error_code-yielding functions can throw exceptions anyway

In LWG discussion of [LWG issue 3014](#), it was pointed out that [\[fs.err.report\]](#) makes several guarantees about the error-reporting behavior of functions in the `std::filesystem` namespace.

Functions **not** having an argument of type `error_code&` handle errors as follows, unless otherwise specified:

- (2.1) When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, an exception of type `filesystem_error` shall be thrown. For functions with a single path argument, that argument shall be passed to the `filesystem_error` constructor with a single path argument. For functions with two path arguments, the first of these arguments shall be passed to the `filesystem_error` constructor as the `path1` argument, and the second shall be passed as the `path2` argument. The `filesystem_error` constructor's `error_code` argument is set as appropriate for the specific operating system dependent error.
- (2.2) **Failure to allocate storage is reported by throwing an exception** as described in [\[res.on.exception.handling\]](#).
- (2.3) Destructors throw nothing.

Functions **having** an argument of type `error_code&` handle errors as follows, unless otherwise specified:

- (3.1) If a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, the `error_code&` argument is set as appropriate for the specific operating system dependent error. Otherwise, `clear()` is called on the `error_code&` argument.

It is unclear whether functions using the "error_code" API (e.g. `std::filesystem::copy_file`) are meant to report "out of memory" errors according to (3.1), or whether the lack of any explicit instruction for "out of memory" errors means that vendors are free to report them by throwing an exception. In practice, it appears that vendors *do* feel free to report "out of memory" conditions via exception-handling.

This arguably sets a precedent that makes the "two-API solution" even harder to teach and use. A programmer might reasonably expect that if an API provides an `error_code&` out-parameter, then *errors will be reported via that out-parameter*. If, as in the case of `std::filesystem`, the presence of an out-parameter cannot be used to detect the error-reporting mechanism — if standard APIs are free to mix several different error-reporting mechanisms within the same function — writing reliable code becomes more challenging.

4.10. Underspecified error_code comparison semantics

Arthur observes that the following program's behavior differs between libc++ and libstdc++:

```
#include <system_error>

int main() {
    std::error_code ec;           // that is, {0, system_category}
    std::error_condition econd;  // that is, {0, generic_category}
    return ec == econd;          // libc++ true, libstdc++ false
}
```

Since error_condition objects are meant only for "catching" (testing) error_code values, perhaps it is incorrect for the programmer to even consider default-constructing an error_condition variable. However, it might still be worthwhile for the Standard to provide guidance in this area.

[Overload journal 141](#) (October 2017) includes an article by Ralph McArdell titled "C++11 (and beyond) Exception Handling," which presents a set of proposed "best practices" for using <system_error>. These best practices basically align with Arthur O'Dwyer's best practices in Section 3 above. A notable difference is that McArdell recommends constructing the "no error" error_code value differently:

```
class ErrCategory : std::error_category { ... };
enum ErrCode { success = 0, failure1 = 1, failure2 = 2 };
template<> struct std::is_error_code_enum : std::true_type {};
inline std::error_code make_error_code(ErrCode e) {
    static const ErrCategory c;
    return std::error_code((int)e, c);
}

std::error_code McArdellNoError() {
    return ErrCode{}; // that is, {0, ErrCategory}
}

std::error_code ODwyerNoError() {
    return std::error_code{}; // that is, {0, system_category}
}
```

A programmer working with one of these libraries, who merely wants to know whether `std::error_code ec` represents an error or not, must have a deep knowledge of all the libraries in his program in order to know whether `!ec` is trustworthy (see point 4.3); and furthermore, he cannot necessarily trust `ec == std::error_condition{}` either (point 4.10); and *furthermore*, if one of his libraries might use McArdell's best practices, he cannot necessarily trust `ec == std::error_code{}` either!